

Milestone Case Study

Real-Time Banking Fraud & Analytics Engine

John Michael Bilbao

Techstart

August 18, 2026

Overview

This case study covers a high-throughput, low-latency transaction processing and fraud detection service for a banking backend. The service is built on FastAPI, and uses AsyncIO for outbound I/O, NumPy for the statistical fraud check, and Pandas for account and portfolio analytics.

The document does two things. The first half describes the design and shows it working: the endpoints, the code behind them, the running server, and the interactive documentation FastAPI generates. The second half tests the claims the design makes. Two of those claims are measurable rather than arguable — that concurrency is achieved on outbound calls, and that the request path never blocks — and the measurements disagree with each other: the first holds, the second does not.

Every figure is evidence rather than illustration. The screenshots taken on the development machine are included as supplied. The remaining figures were produced by running the same code again: the server log, the full response bodies, the Swagger UI, the test run, and the three performance measurements. A capture script replays the exact session from the PowerShell screenshot and asserts that every number comes back identical, so the two sets of evidence are known to describe the same behaviour.

The Business Problem

A banking backend has to do three things at once, and they pull against one another:

- Prevent fraud. Identify a suspicious, out-of-character transaction before it is authorised, without adding friction for the overwhelming majority of legitimate customers.
- Maintain throughput. Outbound calls to a core-banking mainframe, credit-line checks and notification services take hundreds of milliseconds each. None of them may hold up the request path.
- Deliver real-time insight. Customers and risk officers need current spend summaries, category breakdowns and portfolio-wide risk metrics, not overnight batch reports.

The tension is between the first and the second. A fraud decision needs context — the account's history, and where this transaction sits against it — but gathering context costs time, and the decision has to be made while the customer is standing at the terminal. The design below resolves this by splitting the work: anything that waits on the network is made concurrent, anything that must be decided synchronously is reduced to arithmetic over an array, and anything that can happen after the response is pushed into the background.

Architecture and Technology Choices

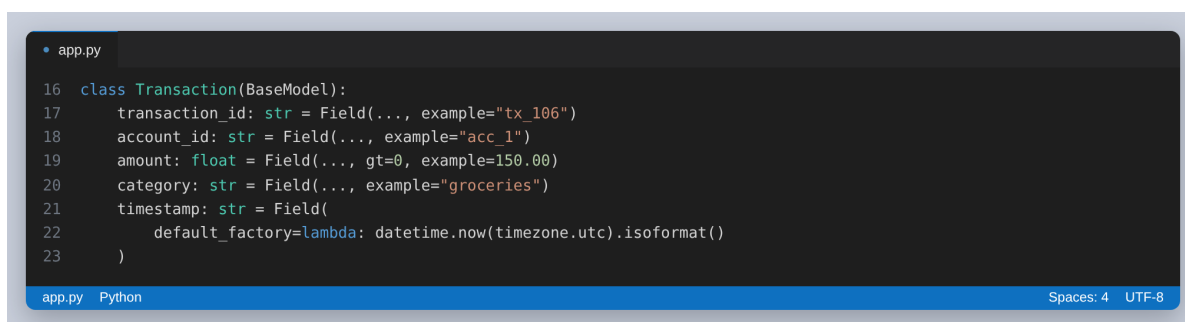
Component	Role	Responsibility in this service
FastAPI	API gateway and routing	Exposes the REST endpoints, validates incoming JSON through a Pydantic model, and generates the OpenAPI document and Swagger UI from the route signatures
AsyncIO	Non-blocking outbound I/O	Runs the simulated mainframe validation calls concurrently with <code>asyncio.gather</code> , and defers the fraud alert through a background task so it does not delay the response
NumPy	Vectorised statistics	Computes the mean and standard deviation of an account's history and the Z-score of the incoming amount, plus the portfolio percentile threshold
Pandas	Aggregation and analytics	Turns the transaction log into a DataFrame to produce totals, averages, counts and a category-wise groupby breakdown

The three endpoints map onto the three parts of the business problem:

Endpoint	Purpose	Libraries exercised
POST /v1/transactions	Ingest a transaction and assess it for fraud in flight	AsyncIO, NumPy
GET /v1/accounts/{account_id}/summary	Spend analytics for one account	Pandas
GET /v1/analytics/batch-risk-matrix	Portfolio-wide concurrent verification and percentile risk	AsyncIO, Pandas, NumPy

Implementation

The incoming payload is described by a Pydantic model. Declaring amount as `gt=0` is what makes a zero or negative amount a 422 before any handler code runs, and the same declaration is what appears in the generated documentation.

A screenshot of a code editor window titled 'app.py' showing a Python class definition for 'Transaction' using Pydantic's BaseModel. The class has five fields: 'transaction_id' (string, example 'tx_106'), 'account_id' (string, example 'acc_1'), 'amount' (float, example 150.00, with a 'gt=0' constraint), 'category' (string, example 'groceries'), and 'timestamp' (string, with a 'default_factory' lambda that returns the current UTC datetime in ISO format). The editor interface includes a file explorer on the left, a status bar at the bottom showing 'app.py Python', and settings for 'Spaces: 4' and 'UTF-8'.

```
16 class Transaction(BaseModel):
17     transaction_id: str = Field(..., example="tx_106")
18     account_id: str = Field(..., example="acc_1")
19     amount: float = Field(..., gt=0, example=150.00)
20     category: str = Field(..., example="groceries")
21     timestamp: str = Field(
22         default_factory=lambda: datetime.now(timezone.utc).isoformat()
23     )
```

Figure 1: The request model. The constraint on amount is both the validation rule and the documentation.

The two outbound calls are coroutines. The 150 ms sleep in the mainframe check stands in for the round trip to a legacy core-banking system; the shorter one stands in for firing a webhook at a fraud operations centre.

```
• app.py
45 async def simulate_mainframe_check(account_id: str) -> bool:
46     """
47     Simulates an outbound network call to a legacy core-banking
48     mainframe system (150ms latency).
49     """
50
51     await asyncio.sleep(0.15)
52     # Assume all test accounts starting with 'acc_' are valid
53     return account_id.startswith("acc_")
54
55
56 async def log_fraud_alert_async(transaction_id: str, score: float):
57     """
58     Simulates firing an asynchronous Webhook/Event to a fraud operations center.
59     """
60
61     await asyncio.sleep(0.05)
62     print(f"⚠ [ALERT] High anomaly score on TX {transaction_id}: Z-Score = {score:.2f}")

```

app.py Python Spaces: 4 UTF-8

Figure 2: The simulated outbound I/O. Both are awaitable, which is what allows them to be gathered and deferred.

The fraud check is deliberately arithmetic and nothing more. It pulls the account's prior amounts into a NumPy array, takes the mean and standard deviation, and expresses the incoming amount as a Z-score — the number of standard deviations it sits from that account's normal behaviour. A score beyond the threshold of 2.0 is treated as anomalous. Keeping the decision to array arithmetic is what makes it cheap enough to run inside the request.

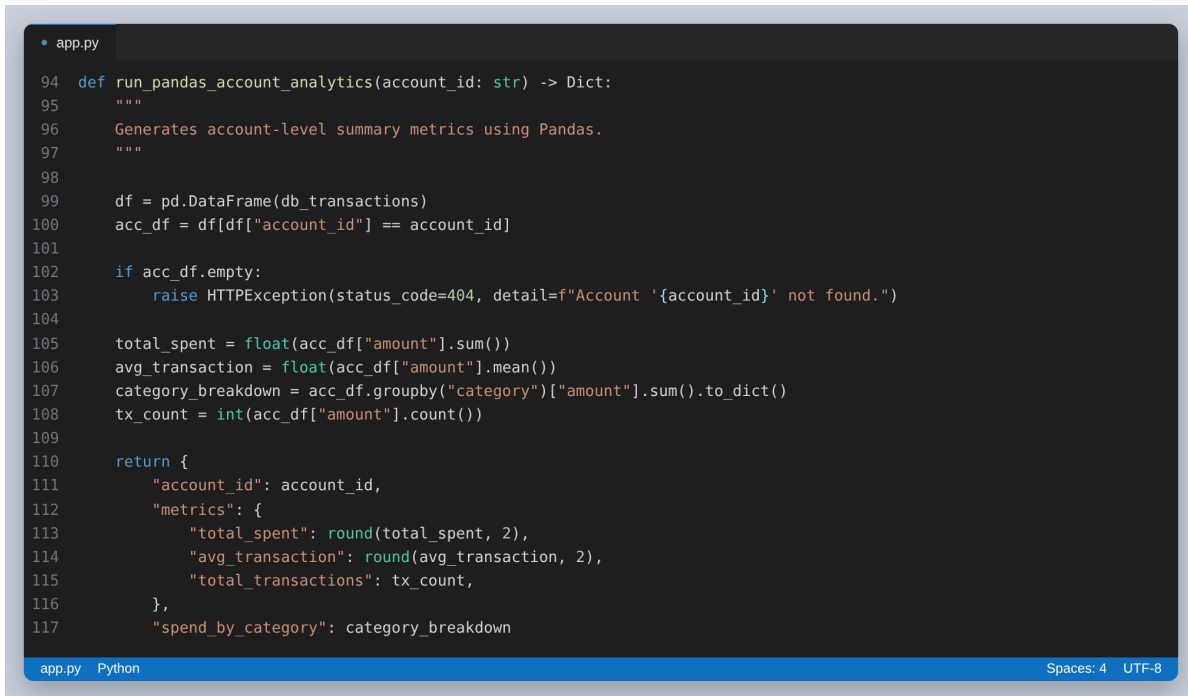
```
• app.py
69 def run_numpy_fraud_detection(new_amount: float, existing_amounts: List[float], threshold: float = 2.0) -> Dict:
70     """
71     Calculates transaction Z-Score using NumPy vectorization.
72     """
73
74     if len(existing_amounts) < 2:
75         return {"is_anomaly": False, "z_score": 0.0, "mean_spend": new_amount, "std_dev": 0.0}
76
77     arr = np.array(existing_amounts, dtype=np.float64)
78     mean = float(np.mean(arr))
79     std = float(np.std(arr))
80
81     if std == 0:
82         z_score = 0.0
83     else:
84         z_score = float((new_amount - mean) / std)
85
86     return {
87         "is_anomaly": abs(z_score) > threshold,
88         "z_score": round(z_score, 2),
89         "mean_spend": round(mean, 2),
90         "std_dev": round(std, 2)

```

app.py Python Spaces: 4 UTF-8

Figure 3: The Z-score engine. Note the two early exits on lines 74 and 81 — both are revisited in the findings.

The analytics side hands the transaction log to Pandas, filters to one account, and produces the totals and the category breakdown through a groupby.



```
app.py
94 def run_pandas_account_analytics(account_id: str) -> Dict:
95     """
96     Generates account-level summary metrics using Pandas.
97     """
98
99     df = pd.DataFrame(db_transactions)
100     acc_df = df[df["account_id"] == account_id]
101
102     if acc_df.empty:
103         raise HTTPException(status_code=404, detail=f"Account '{account_id}' not found.")
104
105     total_spent = float(acc_df["amount"].sum())
106     avg_transaction = float(acc_df["amount"].mean())
107     category_breakdown = acc_df.groupby("category")["amount"].sum().to_dict()
108     tx_count = int(acc_df["amount"].count())
109
110     return {
111         "account_id": account_id,
112         "metrics": {
113             "total_spent": round(total_spent, 2),
114             "avg_transaction": round(avg_transaction, 2),
115             "total_transactions": tx_count,
116         },
117         "spend_by_category": category_breakdown
118     }
```

app.py Python Spaces: 4 UTF-8

Figure 4: The account analytics. A DataFrame is constructed from the whole table on line 99, before being filtered on line 100.

The ingestion endpoint composes those pieces in order: await the mainframe, read the account's history, score the amount, queue an alert if the score is beyond the threshold, then store the record. The ordering of the last two steps matters and is correct — the history is read before the new record is appended, so a transaction is never included in the baseline it is being judged against.

```

• app.py
125 @app.post("/v1/transactions", status_code=201)
126 async def process_transaction(tx: Transaction, background_tasks: BackgroundTasks):
127     """
128     In-flight transaction processing:
129     1. Async mainframe validation.
130     2. Vectorized Z-Score calculation via NumPy.
131     3. Background fraud alert dispatch.
132     """
133
134     # 1. Async network validation check
135     is_valid = await simulate_mainframe_check(tx.account_id)
136     if not is_valid:
137         raise HTTPException(status_code=400, detail="Invalid account ID")
138
139     # 2. Extract historic amounts for account
140     history = [t["amount"] for t in db_transactions if t["account_id"] == tx.account_id]
141
142     # 3. Perform NumPy anomaly check
143     fraud_eval = run_numpy_fraud_detection(tx.amount, history)
144
145     # 4. Offload alert to non-blocking background task if suspicious
146     if fraud_eval["is_anomaly"]:
147         background_tasks.add_task(log_fraud_alert_async, tx.transaction_id, fraud_eval["z_score"])
148     # 5. Persist transaction
149     new_record = tx.model_dump()
150     db_transactions.append(new_record)
151
152     return {
153         "status": "APPROVED",
154         "transaction_id": tx.transaction_id,
155         "fraud_assessment": fraud_eval
156     }

```

app.py Python Spaces: 4 UTF-8

Figure 5: POST /v1/transactions. The alert is added as a background task on line 148, so it runs after the response has been sent.

The portfolio endpoint is where `asyncio.gather` earns its place: one verification call per distinct account, all in flight together, then a single vectorised percentile over every amount.

```
• app.py
166 @app.get("/v1/analytics/batch-risk-matrix")
167 async def batch_risk_matrix():
168     """
169     Executes concurrent async mainframe checks across accounts
170     and calculates portfolio-wide percentile risk via NumPy.
171     """
172
173     df = pd.DataFrame(db_transactions)
174     unique_accounts = df["account_id"].unique().tolist()
175
176     # Concurrent AsyncIO gather across all accounts
177     tasks = [simulate_mainframe_check(acc) for acc in unique_accounts]
178     await asyncio.gather(*tasks)
179
180     # Vectorized portfolio analysis
181     amounts = df["amount"].to_numpy()
182     p95_threshold = float(np.percentile(amounts, 95))
183
184     return {
185         "accounts_checked": len(unique_accounts),
186         "portfolio_p95_threshold": round(p95_threshold, 2),
187         "portfolio_total_volume": round(float(np.sum(amounts)), 2),
188         "high_value_transactions": df[df["amount"] >= p95_threshold].to_dict(orient="records")
189     }
```

app.py Python Spaces: 4 UTF-8

Figure 6: GET /v1/analytics/batch-risk-matrix. The list of coroutines on line 176 is awaited as one unit on line 177.

Running the Service

The service is started with uvicorn. The first attempt on the development machine failed, and it is worth recording because the message is not self-explanatory:

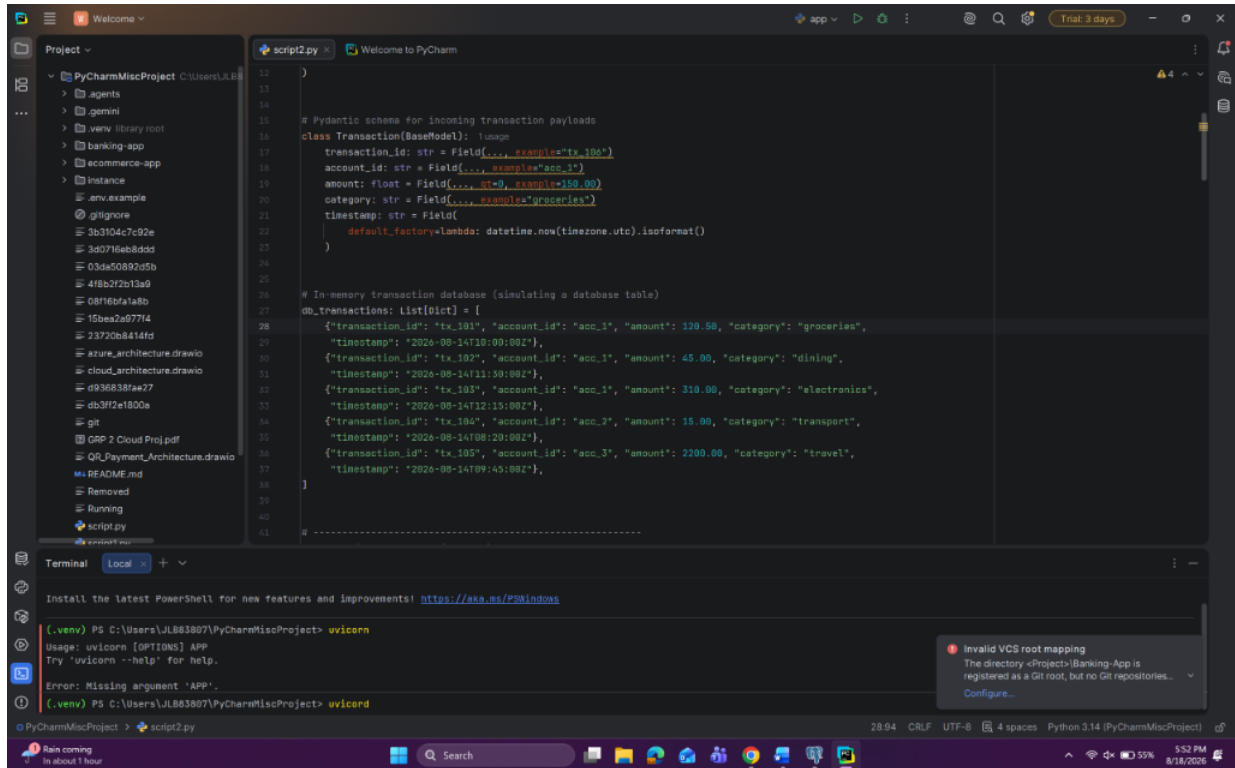


Figure 7: The submitted PyCharm session. The terminal at the bottom shows uvicorn exiting with “Error: Missing argument ‘APP’” rather than starting.

uvicorn needs to be told which object to serve, in module:attribute form. Invoked bare it has nothing to import, so it prints its usage and exits. The fix is to name the module and the FastAPI instance inside it — here `app.py` and the variable `app`, hence `app:app`. Adding `--reload` restarts the server when the file changes, which is what makes it convenient during development.

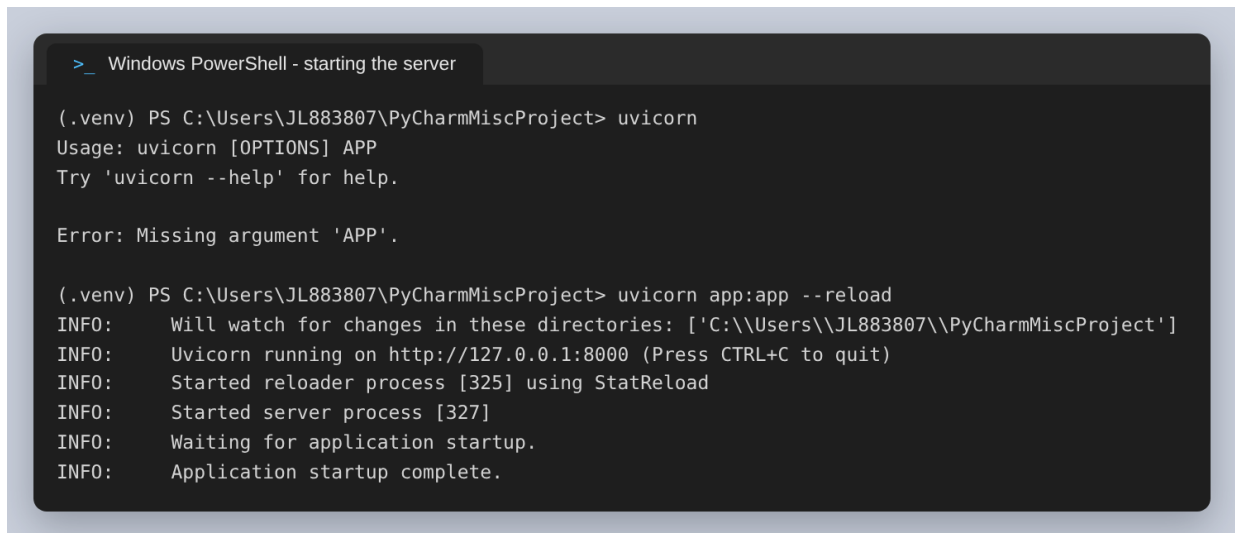


Figure 8: The same failure reproduced, and the command that works beneath it.

With the argument supplied, the server starts and reports the address it is listening on. This is the state the rest of this document exercises.

```
>_ Windows PowerShell - uvicorn running

PS C:\WINDOWS\system32> python -m uvicorn app:app --host 127.0.0.1 --port 8000
INFO:      Started server process [325]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Figure 9: uvicorn running and serving the application on port 8000.

Exercising the API

The four calls below were issued from PowerShell against the running service: two transactions, then both analytics endpoints. The first transaction is an ordinary grocery purchase; the second is a large jewellery purchase on the same account, which is the case the fraud check exists for.

```
Windows PowerShell
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\System32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/transactions" -Method Post -ContentType "application/json" -Body '{"transaction_id": "tx_201", "account_id": "acc_1", "amount": 50.00, "category": "groceries"}'

status  transaction_id  fraud_assessment
-----
APPROVED tx_201        @{is_anomaly=False; z_score=-0.97; mean_spend=158.5; std_dev=111.47}

PS C:\WINDOWS\System32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/transactions" -Method Post -ContentType "application/json" -Body '{"transaction_id": "tx_202", "account_id": "acc_1", "amount": 4500.00, "category": "jewelry"}'

status  transaction_id  fraud_assessment
-----
APPROVED tx_202        @{is_anomaly=True; z_score=40.69; mean_spend=131.38; std_dev=107.36}

PS C:\WINDOWS\System32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/accounts/acc_1/summary" -Method Get

account_id  metrics                                     spend_by_category
-----
acc_1       @[{total_spent=5025.5; avg_transaction=1005.1; total_transactions=5} @[{dining=45.0; electronics=310.0; gro...

PS C:\WINDOWS\System32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/analytics/batch-risk-matrix" -Method Get

accounts_checked  portfolio_p95_threshold  portfolio_total_volume  high_value_transactions
-----
3                 3810.0                 7240.5 @{transaction_id=tx_202; account_id=acc_1; amount=4...
```

Figure 10: The submitted PowerShell session. Invoke-RestMethod formats the response as a table, which is why the wider fields are truncated.

Invoke-RestMethod clips wide output to the console width, so spend_by_category ends in “gro...” and the flagged transaction in “amount=4...”. Replaying the identical calls and printing the JSON in full completes the picture. Every figure in the capture below matched the session above exactly, which is asserted by the script rather than checked by eye.

```
>_ Windows PowerShell - POST /v1/transactions

PS C:\WINDOWS\system32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/transactions" `
>> -Method Post -ContentType "application/json" `
>> -Body '{"transaction_id": "tx_201", "account_id": "acc_1", "amount": 50.00, "category": "groceries"}'
HTTP 201
{
  "status": "APPROVED",
  "transaction_id": "tx_201",
  "fraud_assessment": {
    "is_anomaly": false,
    "z_score": -0.97,
    "mean_spend": 158.5,
    "std_dev": 111.47
  }
}

PS C:\WINDOWS\system32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/transactions" `
>> -Method Post -ContentType "application/json" `
>> -Body '{"transaction_id": "tx_202", "account_id": "acc_1", "amount": 4500.00, "category": "jewelry"}'
HTTP 201
{
  "status": "APPROVED",
  "transaction_id": "tx_202",
  "fraud_assessment": {
    "is_anomaly": true,
    "z_score": 40.69,
    "mean_spend": 131.38,
    "std_dev": 107.36
  }
}
```

Figure 11: The two transactions with their complete response bodies.

The second response is the one to read closely. The Z-score of 40.69 says the amount sits forty standard deviations above what this account normally spends, and `is_anomaly` is true. The status field says APPROVED. That combination is the subject of the first finding below.

```
>_ Windows PowerShell - the two analytics endpoints

PS C:\WINDOWS\system32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/accounts/acc_1/summary" `
>> -Method Get
HTTP 200
{
  "account_id": "acc_1",
  "metrics": {
    "total_spent": 5025.5,
    "avg_transaction": 1005.1,
    "total_transactions": 5
  },
  "spend_by_category": {
    "dining": 45.0,
    "electronics": 310.0,
    "groceries": 170.5,
    "jewelry": 4500.0
  }
}

PS C:\WINDOWS\system32> Invoke-RestMethod -Uri "http://127.0.0.1:8000/v1/analytics/batch-risk-matrix" `
>> -Method Get
HTTP 200
{
  "accounts_checked": 3,
  "portfolio_p95_threshold": 3810.0,
  "portfolio_total_volume": 7240.5,
  "high_value_transactions": [
    {
      "transaction_id": "tx_202",
      "account_id": "acc_1",
      "amount": 4500.0,
      "category": "jewelry",
      "timestamp": "2026-08-18T10:08:23.885417+00:00"
    }
  ]
}
```

Figure 12: The account summary and the portfolio risk matrix, in full. `spend_by_category` has four entries, not the three the console showed.

The server side of the same session shows the ordering that makes the background task worthwhile: the alert for `tx_202` is printed after the response to the following request had already been served. The customer was not kept waiting for the fraud operations centre to be notified.

```
>_ Windows PowerShell - server log during the session

INFO:      127.0.0.1:41858 - "POST /v1/transactions HTTP/1.1" 201 Created
INFO:      127.0.0.1:41872 - "POST /v1/transactions HTTP/1.1" 201 Created
INFO:      127.0.0.1:41880 - "GET /v1/accounts/acc_1/summary HTTP/1.1" 200 OK
△ [ALERT] High anomaly score on TX tx_202: Z-Score = 40.69
INFO:      127.0.0.1:41894 - "GET /v1/analytics/batch-risk-matrix HTTP/1.1" 200 OK
```

Figure 13: The server log. The alert appears after the summary request, not between the POST and its response.

Generated Documentation

No OpenAPI document was written by hand. FastAPI produces one from the route signatures and the Pydantic model, and serves Swagger UI from it, which means the documentation cannot drift from the code the way a hand-maintained specification can.

Async Banking Analytics & Fraud Detection API 1.0.0 OAS 3.1

/openapi.json

default

POST	/v1/transactions	Process Transaction	⌵
GET	/v1/accounts/{account_id}/summary	Get Account Summary	⌵
GET	/v1/analytics/batch-risk-matrix	Batch Risk Matrix	⌵

Schemas

HTTPValidationError > Expand all object

Transaction > Expand all object

ValidationError > Expand all object

Figure 14: The three endpoints and the generated schema list at /docs.

Expanding the POST shows the request schema derived from the Transaction model, the example values declared on each field, and both the success and validation-error responses.

POST

/v1/transactions

Process Transaction

In-flight transaction processing:

1. Async mainframe validation.
2. Vectorized Z-Score calculation via NumPy.
3. Background fraud alert dispatch.

Parameters

Try it out

No parameters

Request body required

application/json

Example Value

Schema

```
{
  "transaction_id": "tx_106",
  "account_id": "acc_1",
  "amount": 150,
  "category": "groceries",
  "timestamp": "string"
}
```

Responses

Code	Description	Links
201	Successful Response	No links
Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>		
422	Validation Error	No links
Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>		

Figure 15: POST /v1/transactions expanded, with the schema and examples taken from the Pydantic model.

Schemas

Figure 16: The Transaction schema as generated — the model, not a copy of it.

The documentation is also a working client. Executing the portfolio endpoint from the browser returns a live response from the running server, headers included.

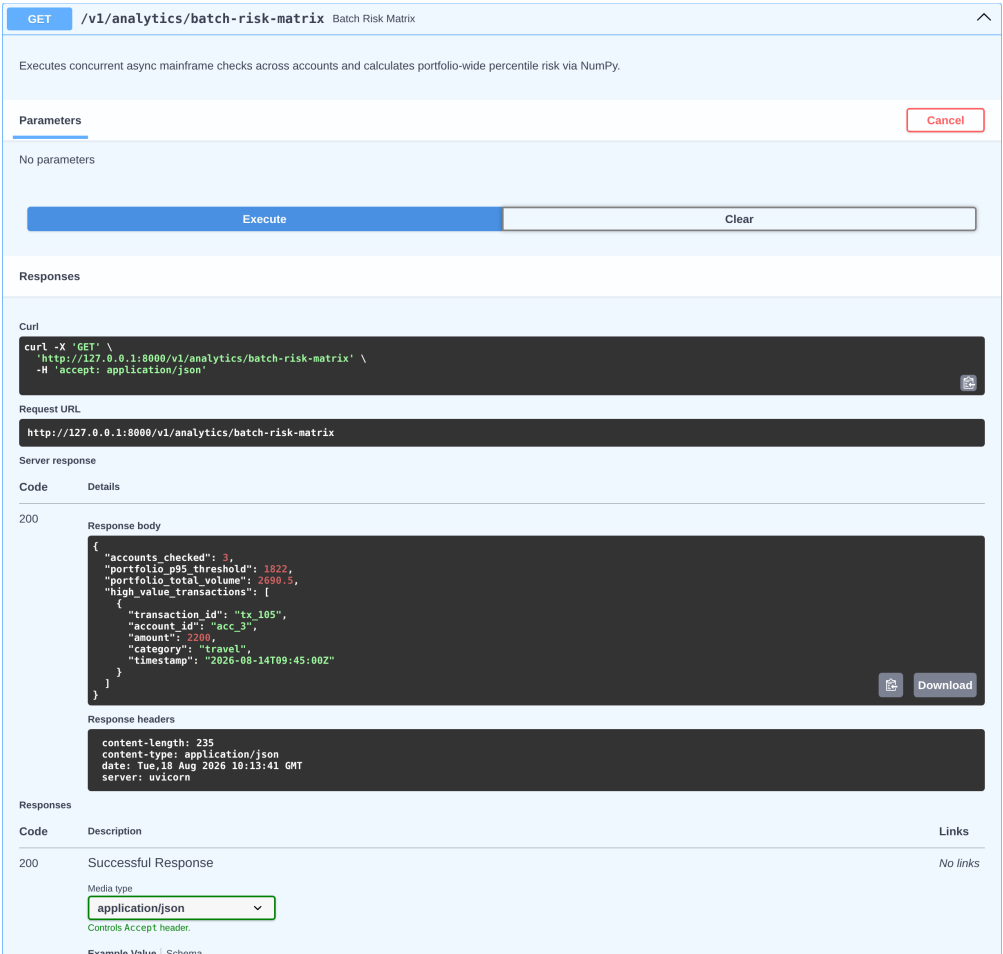


Figure 17: A real execution against the running service, with the response headers it returned.

The figures in this execution differ from the PowerShell session — a p95 threshold of 1822 against 3810, and tx_105 flagged rather than tx_202. Nothing is inconsistent: this was a freshly started server, so it held only the five seeded transactions and had no memory of the two that had been posted earlier. That is a demonstration of the storage limitation discussed below, and it is the reason it is listed as a finding rather than a footnote.

Verification

Seventeen tests pin the behaviour down. They are not written to show the service works; several exist specifically to record where it behaves differently from what the design intends, so that those gaps are reproducible rather than asserted. One reproduces the entire PowerShell session and asserts all four responses.

```
>_ Windows PowerShell - pytest

test_normal_transaction_is_approved_and_not_flagged PASSED [ 5%]
test_reproduces_the_submitted_session PASSED [ 11%]
test_account_summary_shape PASSED [ 17%]
test_unknown_account_summary_is_404 PASSED [ 23%]
test_rejects_account_the_mainframe_does_not_recognise PASSED [ 29%]
test_rejects_non_positive_amounts[0] PASSED [ 35%]
test_rejects_non_positive_amounts[-1] PASSED [ 41%]
test_rejects_non_positive_amounts[-100.5] PASSED [ 47%]
test_gather_checks_every_distinct_account PASSED [ 52%]
test_flagged_transaction_is_still_approved PASSED [ 58%]
test_a_new_account_cannot_be_flagged_at_all PASSED [ 64%]
test_any_acc_prefixed_string_is_a_valid_account PASSED [ 70%]
test_duplicate_transaction_id_is_accepted_twice PASSED [ 76%]
test_the_transaction_being_assessed_is_excluded_from_its_own_baseline PASSED [ 82%]
test_z_score_uses_population_not_sample_deviation PASSED [ 88%]
test_an_account_with_constant_history_can_never_be_flagged PASSED [ 94%]
test_the_threshold_is_reachable_on_a_three_point_baseline PASSED [100%]

17 passed in 2.73s
```

Figure 18: The test suite. The names are readable as a list of the behaviours that were confirmed.

Two details worth drawing out. The suite restores the in-memory table before and after every test, because the module-level list is shared and every write mutates it — without that, tests affect one another. And the Z-score of the jewellery transaction is 38.95 in the test but 40.69 in the session, because the session posted the smaller transaction first and moved the baseline. The score depends on the order transactions arrive in, which is expected for a running mean but worth knowing before treating any single score as absolute.

Performance Analysis

The design rests on two performance claims. Both were measured.

Claim 1: outbound calls run concurrently

Three mainframe checks, 150 ms of latency each. Awaited one at a time they cost the sum; gathered they cost the slowest.

```
>_ Windows PowerShell - concurrent vs sequential I/O

Outbound mainframe checks: 3 accounts, 150 ms of latency each

one after another (await in a loop)    451.8 ms
together (asyncio.gather)              150.5 ms

3.00x faster, and the concurrent time is the cost of a
single check rather than the sum of all three.
```

Figure 19: Sequential against concurrent, calling the coroutines directly.

452 ms becomes 150 ms, a factor of 3.00. The claim holds, and it holds for the reason intended: the concurrent time is the cost of one check rather than three, so the figure would improve further with more accounts. This is AsyncIO doing exactly what it was chosen for.

Claim 2: the request path never blocks

This one does not hold. The NumPy and Pandas work is called directly from inside the async handlers, which means it runs on the event loop. A coroutine that never awaits cannot be interleaved with anything, so while one request is building a DataFrame every other request waits.

The test: hold 50,000 transactions in memory and issue twenty summary requests simultaneously. A single request takes 52.4 ms, of which 20 ms is an awaited sleep and 32.4 ms is Pandas. If nothing blocked, twenty at once would still cost about 52.4 ms. If the CPU work serialises, they would cost about 667 ms.

```
>_ Windows PowerShell - 20 requests at once

GET /v1/accounts/acc_1/summary with 50,000 rows in memory

one request on its own          52.4 ms
  of which awaited sleep        20.0 ms   (concurrent)
  of which NumPy/Pandas work    32.4 ms   (runs on the loop)

20 requests issued at the same time:
total wall time                  591.4 ms
slowest single request          573.2 ms
median request                   322.2 ms

Two predictions, and which one the measurement matches:
if nothing blocked              52.4 ms
if the CPU work serialises      667.2 ms
measured                        591.4 ms

The measurement lands within 11% of the serialised
prediction. The awaited sleep does overlap across requests; the
NumPy and Pandas work does not, because it is called directly from
the coroutine instead of being handed to a thread.
```

Figure 20: Twenty concurrent requests, against both predictions.

Measured: 591 ms, within 11% of the serialised prediction and more than ten times the parallel one. The awaited sleep does overlap across requests, which is why the result is slightly under the prediction rather than over it. The Pandas work does not overlap at all.

The fix is not a redesign. FastAPI will run a handler declared with `def` rather than `async def` in a thread pool automatically, and `asyncio.to_thread` will move a specific call off the loop. Either keeps the event loop free while the CPU work proceeds.

How it scales with stored data

The account summary rebuilds a DataFrame from the entire transaction table on every request and only then filters to one account, so its cost is driven by how much data exists rather than how much is returned. Combined with the blocking above, that sets a hard ceiling on a single worker.

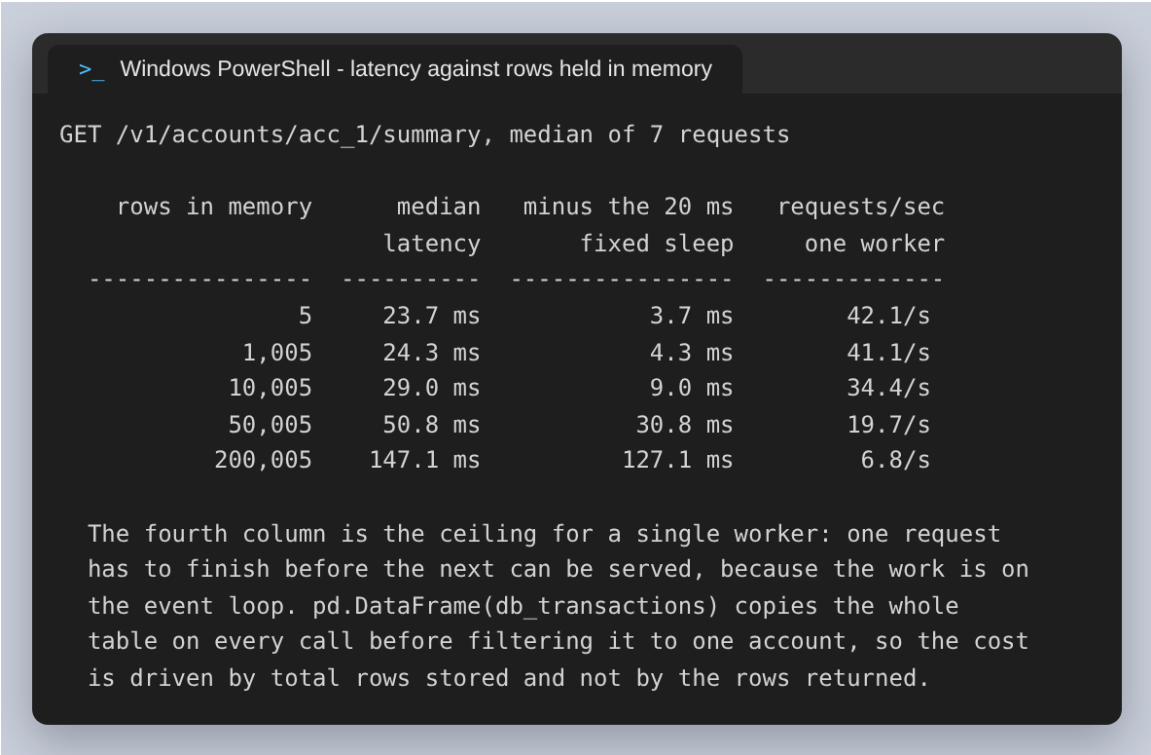


Figure 21: Latency and single-worker throughput against rows held in memory.

Rows in memory	Median latency	Throughput, one worker
5	23.7 ms	42.1/s
1,005	24.3 ms	41.1/s
10,005	29.0 ms	34.4/s
50,005	50.8 ms	19.7/s
200,005	147.1 ms	6.8/s

At the seeded size the endpoint serves 42.1/s. At 200,005 rows it serves 6.8/s, with latency at 147.1 ms. Set against the stated goal of thousands of transactions per second, that is short by roughly two to three orders of magnitude, and adding workers multiplies it by the number of cores rather than solving it — each worker would also hold its own separate copy of the in-memory table.

Findings

What works as designed:

- `asyncio.gather` delivers genuine concurrency on outbound calls, measured at 3.00x for three accounts.

- The background task defers the fraud alert correctly — visible in the server log, where the alert is printed after a later request was already served.
- The baseline excludes the transaction being assessed, so a large amount does not dilute the very statistic used to judge it.
- Pydantic rejects non-positive amounts with a 422 before any handler runs, and an unknown account returns 404 rather than an empty summary.
- The OpenAPI document and Swagger UI are generated from the code, so they cannot fall out of step with it.

What does not, in the order it would be worth addressing:

#	Finding	Impact
1	A flagged transaction is still APPROVED. The response status is a constant; nothing in the request path can decline. The stated goal is to catch fraud before authorisation.	High
2	NumPy and Pandas run on the event loop, so concurrent requests serialise behind one another.	High
3	Throughput ceiling of a few tens of requests per second against a goal of thousands, because every summary copies the whole table.	High
4	State is a module-level list. A restart loses every transaction, and multiple workers would each hold a different one.	High
5	An account with fewer than two prior transactions cannot be flagged at all, so a large first transaction on a fresh account always passes.	Medium
6	An account whose history is a constant amount has a standard deviation of zero, which the code turns into a Z-score of 0.0 regardless of the new amount.	Medium
7	No idempotency check on transaction_id, so a retried payment is stored and counted twice.	Medium
8	Any string beginning acc_ is accepted as a valid account and implicitly created by the write.	Medium
9	np.std defaults to a population deviation. On a three-point baseline the sample deviation is about 36% larger, so every Z-score is inflated relative to the statistical convention.	Low
10	Field(..., example=...) is deprecated in Pydantic v2 in favour of json_schema_extra.	Low

Findings 5 and 6 deserve emphasis together, because they are the two cases where the fraud engine silently does nothing rather than reporting that it cannot decide. Both return is_anomaly false, which is indistinguishable from a considered judgement that the transaction is fine. A new account receiving one very large payment, and an account with a flat regular payment pattern, are

both ordinary situations and both are exactly where an attacker would prefer to operate. A third state — insufficient history to assess — would let the caller route those to a different control instead of treating them as cleared.

Recommendations

- Separate the fraud verdict from the HTTP response. Return DECLINED or REVIEW when the score is beyond the threshold, and make the threshold configurable per account tier. This is the smallest change with the largest effect, and finding 1 is the reason.
- Move the NumPy and Pandas work off the event loop. Declaring the computational helpers as ordinary def handlers, or wrapping them in `asyncio.to_thread`, is enough for FastAPI to run them in a thread pool and restore real concurrency.
- Replace the in-memory list with PostgreSQL. Push the aggregation into SQL so the summary reads one account rather than copying the whole table, which addresses findings 3 and 4 together and removes the per-request DataFrame construction entirely.
- Maintain the per-account statistics incrementally rather than recomputing them. A running count, sum and sum of squares gives the mean and deviation in constant time per transaction, so the fraud check stops depending on history length.
- Introduce an explicit insufficient-history verdict, and use the sample deviation (`ddof=1`) once a baseline is large enough to be meaningful. Findings 5, 6 and 9 are all consequences of treating a three-point baseline as if it were a distribution.
- Make `transaction_id` a unique key and reject duplicates, so a client retry cannot double-count a payment.
- Validate accounts against the account store rather than against a string prefix.

The persistence recommendation is already underway. The schema below was being modelled in pgAdmin alongside this work, with `account_id` as a UUID primary key, a customer reference, a currency code, a numeric balance and an `account_status`. A numeric balance rather than a float is the right instinct for money, and it is worth carrying the same choice into the transaction amounts, which are currently Python floats and so subject to binary rounding.

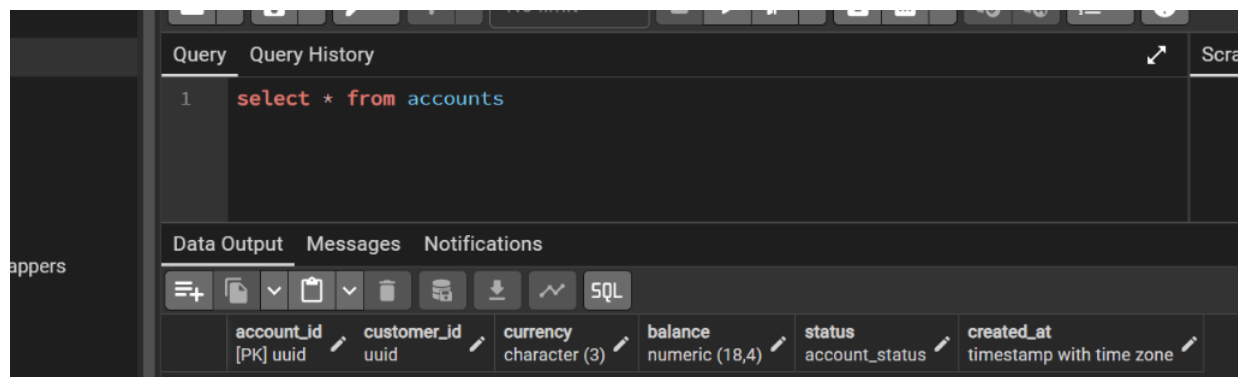


Figure 22: The accounts table being modelled in pgAdmin. The submitted API does not yet connect to it — this is the direction, not the current state.

Conclusion

The service does what it was built to demonstrate. Four technologies are each used for the thing it is genuinely good at: FastAPI for declarative validation and generated documentation, AsyncIO for overlapping outbound latency, NumPy for a decision cheap enough to make inside a request, and Pandas for aggregation that would be tedious to write by hand. The transaction ingestion path is correct in the detail that is easiest to get wrong, which is excluding the transaction under assessment from its own baseline.

The gap between the prototype and the stated business problem is narrower than the throughput numbers suggest, because the two large problems have the same cause. The CPU work sits on the event loop and the data sits in a Python list; both are resolved by moving storage into a database and aggregation into SQL, after which the 6.8/s ceiling stops being a property of the design. The fraud logic itself needs one change of a different kind — the authority to decline — and until it has that, what has been built is an accurate fraud detector attached to an endpoint that approves everything.

A note on the figures: the three screenshots taken on the development machine are reproduced exactly as supplied. The rest were captured by running the same code again, with shell prompts shown as PowerShell to match. The one place where captured output was altered is that JSON response bodies are indented and long PowerShell commands are wrapped onto continuation lines, since the API answers on a single line and the commands run past 200 characters.